ใบเตย

PANDAN LEAF

THAI KITCHEN

Restaurant Ordering System

A full-stack QR-code ordering, kitchen and payment platform

Project overview · architecture · features · structure

github.com/BhonePyaeMin/Restaurant-Ordering-System

Project Overview

ใบเตย (“Pandan Leaf”) is a working, end-to-end restaurant ordering system built as a portfolio / learning project. A guest scans a QR code at their table, orders from a bilingual Thai/English menu on their own phone, and watches the order move live through **Received** → **Cooking** → **Ready** → **Served**. The kitchen sees every new order the moment it's placed, with a countdown per dish that moves the ticket to “Ready” on its own once the slowest dish's cook time is up. The table's shared bill updates in real time for everyone sitting there, and can be settled by cash, card, or a real PromptPay QR code. The owner has a dashboard for sales, live orders and tables, the menu, and printable table QR codes.

How it flows

Customer scans a QR → order.html?table=N → the C# API saves the order → it tells the Node server → Node pushes it to the kitchen screen instantly, with a sound. Kitchen taps “Start cooking” (or the per-dish timer runs out) → Node → the C# API updates the order → the customer's phone updates live, with no refresh needed. The C# API is the single source of truth for the menu, tables, orders and money; the Node server stores nothing of its own — it only relays events in real time over Socket.IO.

Tech stack

Layer	Technology	Role
API	C# · ASP.NET Core 8 · EF Core · SQLite	Menu, orders, tables, payments, admin dashboard, the cook-time auto-Ready service
Real-time server	Node.js · Express · Socket.IO	Pushes order and bill updates to kitchen screens and customer phones instantly
Front end	Plain HTML · CSS · JavaScript (no framework)	Customer ordering page, admin dashboard, kitchen display — all bilingual TH/EN
Payments	PromptPay (Thailand) QR · Cash · Card	Real EMVCo/PromptPay QR codes generated server-side, plus a safe demo mode

Core features

- **QR ordering** — each table has its own QR code; prices always come from the database, never trusted from the phone.
- **Live order tracking** — a step tracker (Received → Cooking → Ready → Served) that updates over a socket, with a 20-second poll as a fallback if the connection drops.
- **Per-dish cook timers** — every dish has a prep time; a background service moves an order to “Ready” automatically once the slowest dish's timer is up, while staff can still mark it ready early.
- **Shared table bills** — everyone at a table sees one live bill, even a phone that ordered nothing itself; guests can request the bill or pay by Cash, Card, or a real PromptPay QR (with a clearly-marked demo mode when no PromptPay account is configured).
- **Kitchen display** — PIN-protected, three columns (New / Cooking / Ready), a sound on new orders, a “who wants the bill” strip, and smooth scrolling.

- **Admin dashboard** — username/password login, today's revenue and top dishes, a live table grid, full order history, menu management (add/edit/sold-out), and printable QR codes including one general “counter” PromptPay QR.
- **Bilingual throughout** — a ไทย/EN switch on the customer and admin pages; the kitchen display uses clear plain-English column names.
- **Safety by design** — admin actions are rate-limited and password-protected, order status can only move forward, and a demo PromptPay code is clearly labelled so it can never be mistaken for a real payment method.

Project Structure

The repository is organized as two independent services sharing one database of truth (the C# API), plus the static front-end pages the API serves.

```
api/ — C# ASP.NET Core 8 · EF Core + SQLite · port 5000
|-- Program.cs setup, database creation, static files, the auto-ready background job
|-- appsettings.json admin login, PromptPay settings, kitchen URL + secret, time zone
|-- Models/Entities.cs Category, MenuItem, RestaurantTable, Order, OrderItem
|-- Data/ AppDbContext + SeedData (the Pandan Leaf Thai menu, tables 1–10)
|-- Dtos/ request/response shapes with validation + mapping
|-- Controllers/ Menu, Orders, Tables (public bill + PromptPay QR),
| Admin (dashboard, tables, payment, QR), Auth (login)
|-- Services/ KitchenNotifier, AdminKey filter, RestaurantClock, PromptPayQr,
| TableBills, AutoReadyService (cook-timer → auto Ready)
`-- wwwroot/ index.html, order.html, admin.html, css/theme.css, icons

kitchen-server/ — Node.js · Express + Socket.IO · port 4000
|-- server.js receives events from the API, broadcasts them, forwards kitchen taps
|-- .env.example secrets + settings (PIN, admin key, internal secret)
`-- public/index.html the kitchen display screen

PROMPTS.md — copy-paste AI prompts to build or extend the project step by step
README.md — full setup, run, and API reference
```

Order status rules

An order only ever moves forward: **Pending** → **Preparing** → **Ready** → **Served**, or **Cancelled** while still Pending or Preparing. Any other transition is rejected with a 409 Conflict. A table can only be paid once every one of its orders is Served (or Cancelled); once paid, an order can't change again. If the Node server is ever down, orders are still saved by the API and the pages fall back to polling every 20 seconds.

Brand

Navy #3D405B	Headers and main text
Terracotta #E07A5F	Main buttons, prices, the “Cooking” column
Sage #81B29A	“Ready” / “Available” labels, the logo
Sand #F2CC8F	“New” orders and small highlights
Cream #F7F4EA	Page background
Night #1E2033	Kitchen screen background

Fonts: Noto Sans Thai for text, Noto Serif Thai for headings — loaded from Google Fonts, with a system-font fallback when offline.

Links

Repository: **github.com/BhonePyaeMin/Restaurant-Ordering-System**

Local URLs when running: Start page http://localhost:5000 · Admin http://localhost:5000/admin.html · Kitchen http://localhost:4000 · API docs http://localhost:5000/swagger

Generated as a project overview for the ใบเตย · Pandan Leaf Restaurant Ordering System.